

List Comprehensions and Multi-Record Processing Patterns

Turn yesterday's loops into compact transformations, then assemble a small pipeline.

Today's mission

Create new collections from old ones.
Filter records.
Transform fields.
Separate clean and rejected output.



Pattern: input → validation → transformation → output

Why this block matters

Most useful programs transform collections of data, not just one value.

```
raw_statuses = [" ACTIVE ", "inactive",  
"Active"]  
  
normalized_statuses = []  
  
for status in raw_statuses:  
    normalized_statuses.append(status.strip().lower())
```

We often need to create a new collection while keeping the original input unchanged.



Do not overwrite raw data until you know your transformation is correct.

Traditional transformation loop

The long-form loop makes every step visible.

```
numbers = [1, 2, 3, 4]
squares = []
for number in numbers:
    squares.append(number ** 2)
print(squares)
```

1. Create output list

2. Visit each input value

3. Transform current value

4. Append result

Result: [1, 4, 9, 16]

Equivalent list comprehension

A comprehension compresses a common loop pattern into one expression.

```
squares = [  
    number ** 2  
    for number in numbers  
]
```

Same idea:

Take each number from numbers, square it, and place the result in a new list.

```
squares = [number ** 2 for number in numbers]
```

Readable is the goal. Compact is a bonus.

How to read a comprehension

Read it as: “make a list of this expression for each item in that source.”

```
new_list = [  
    expression  
    for item in source_collection  
]
```

expression

What gets placed into the output list.

item

The temporary name for one current value.

source_collection

The input values being processed.

A comprehension creates a new list. It does not automatically change the old list.

Filtering collections

Filtering means keeping only values that pass a test.

```
scores = [82, 61, 74, 95, 55, 88]

passing_scores = []

for score in scores:
    if score >= 70:
        passing_scores.append(score)
```

```
passing_scores = [
    score
    for score in scores
    if score >= 70
]
```

Condition at the end decides what gets kept.

Transform and filter

A comprehension can both choose values and transform them.

```
temperatures = [72, 51, 81, 68]

normal_temperatures = [
    temperature
    for temperature in temperatures
    if 60 <= temperature <= 80
]
```

The source list is unchanged.

The output list contains only
temperatures inside the normal
range.

Input: 72, 51, 81, 68



Output: 72, 68

Extracting fields from records

When records are dictionaries, a comprehension can pull out one field.

```
records = [  
    {"record_id": "EQ-1001", "status": "active"},  
    {"record_id": "EQ-1002", "status":  
"inactive"},  
]  
  
record_ids = [  
    record["record_id"]  
    for record in records  
]
```

list of dictionaries



one field per record



new list of IDs

This is a very common API, CSV, and JSON pattern.

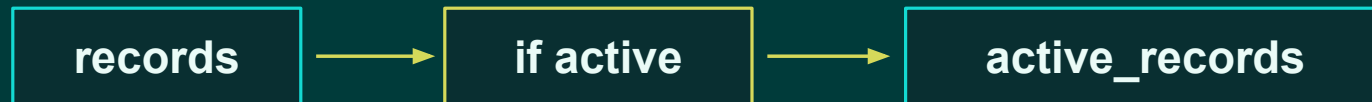
Filtering dictionaries

Keep whole records when the record meets a condition.

```
active_records = [  
    record  
    for record in records  
    if record["status"] == "active"  
]
```

Notice the expression is just record.

That means the entire dictionary is kept, not only one field.



Filter = smaller collection with the same kind of item.

When not to use a comprehension

A compact expression is not automatically better.

```
results = [  
    complicated_transformation(record)  
    for record in records  
    if first_condition(record)  
    and second_condition(record)  
    and third_condition(record)  
]
```

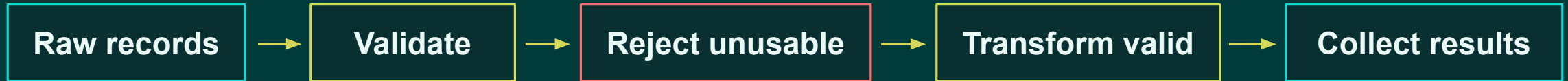
This may technically work, but it hides too much decision logic.

- Use a normal loop when logic has multiple branches.
- Use named Boolean variables to explain rules.
- Prefer debugging clarity over clever code.

Rule: use comprehensions only when the operation stays immediately understandable.

Multi-record processing pattern

Data processing is often a sequence of stages.



Keep the raw input separate from the cleaned output so you can audit what changed.

This is the shape of many real data pipelines.

Pipeline loop: split clean and rejected records

A normal loop is the clearest choice for multi-step processing.

```
clean_records = []
rejected_records = []

for record in records:
    if record["record_id"] == "":
        rejected_records.append(record)
        continue

    cleaned_record = {
        "record_id":
record["record_id"].strip().upper(),
        "status": record["status"].strip().lower(),
    }

    clean_records.append(cleaned_record)
```

Why a loop here?

- We branch
- We skip records
- We build new dictionaries
- We update two output collections

Comprehensions are tools, not a requirement.

Guided exercise 1: squares

Write the long form first, then the comprehension.

```
numbers = [1, 2, 3, 4, 5]
```

Part A

Use a traditional loop to create squares.

Part B

Create the same result with a list comprehension.

Expected result: [1, 4, 9, 16, 25]

Exercise time: 5 minutes

Guided exercise 2: filter and normalize

Practice both filtering and transforming.

```
scores = [82, 61, 74, 95, 55, 88]

statuses = [
    " ACTIVE ",
    "inactive",
    "Active",
    " RETIRED ",
]
```

Create:

passing_scores
normalized_statuses

- Passing means score ≥ 70 .
- Normalize with strip() and lower().
- Use comprehensions where clear.

Exercise time: 8 minutes

Guided exercise 3: extract record IDs

Create a list of fields from a list of dictionaries.

```
records = [  
    {"record_id": "EQ-1001", "status": "active"},  
    {"record_id": "EQ-1002", "status":  
"inactive"},  
    {"record_id": "EQ-1003", "status": "active"},  
]
```

Create `record_ids` using a list comprehension.

```
record_ids = [  
    ...  
    for ... in ...  
]
```

Expected: ["EQ-1001", "EQ-1002", "EQ-1003"]

Guided exercise 4: active record IDs

Combine field extraction and filtering.

```
active_record_ids = [  
    record["record_id"]  
    for record in records  
    if record["status"] == "active"  
]
```

Your job:

Explain each line in plain language before running it.

extract this field



from each record



only if active

Exercise time: 6 minutes

Applied challenge: raw records

This is the data intake problem for the end of the block.

```
raw_records = [  
    {"record_id": " eq-1001 ", "status": " ACTIVE ",  
     "error_count": 0, "completion": 100},  
    {"record_id": "", "status": "active",  
     "error_count": 0, "completion": 85},  
    {"record_id": "EQ-1003", "status": " INACTIVE ",  
     "error_count": 0, "completion": 100},  
    {"record_id": "eq-1004", "status": "active",  
     "error_count": 4, "completion": 95},  
]
```

Outputs to build:

clean_records
rejected_records
summary statistics

Challenge time: 20 minutes

Challenge scaffold: normalize and reject

Start by creating output collections and safe normalized values.

```
clean_records = []
rejected_records = []

for record in raw_records:
    record_id = record["record_id"].strip().upper()
    status = record["status"].strip().lower()

    if record_id == "":
        rejected_records.append(record)
        continue

    if status == "inactive":
        rejected_records.append(record)
        continue
```

First rule:
reject missing IDs.

Second rule:
reject inactive records.

**continue prevents bad records from
reaching later processing.**

Challenge scaffold: assign status and append

Cleaned output records should be consistent and easy to read.

```
processing_status = "ready"

if record["error_count"] > 0:
    processing_status = "warning"

if record["completion"] < 100:
    processing_status = "warning"

cleaned_record = {
    "record_id": record_id,
    "status": status,
    "error_count": record["error_count"],
    "completion": record["completion"],
    "processing_status": processing_status,
}

clean_records.append(cleaned_record)
```

The new dictionary is the cleaned version of the raw record.

- Normalized ID
- Normalized status
- Warning for errors
- Warning for incomplete records

Append exactly once per accepted record.

Challenge scaffold: summary statistics

After processing, summarize the results.

```
total_records = len(raw_records)
clean_count = len(clean_records)
rejected_count = len(rejected_records)

warning_count = 0

for record in clean_records:
    if record["processing_status"] == "warning":
        warning_count += 1

ready_count = clean_count - warning_count
```

Display:

- Total input records
- Clean records
- Rejected records
- Ready records
- Warning records

Summary numbers prove your pipeline behaved as expected.

Common mistakes

Most errors come from mixing source and output or hiding too much logic.

Mistake 1

Appending to the same list
you are looping over.

Mistake 2

Using a comprehension for
complicated branching.

Mistake 3

Forgetting to assign method
results like `strip()` or `lower()`.

```
status.strip().lower()      # result is  
ignored  
status = status.strip().lower() # corrected
```

Debug strategy: print one record, one field, one decision at a time.

Knowledge check and bridge

Students should be able to explain these patterns in plain language.

Transform

Create a new value from each input item.

Filter

Keep only items that pass a condition.

Pipeline

Separate input, validation, transformation, and output.

Next up: functions.

We will package repeated logic so the same validation or transformation can be reused cleanly.

Core idea: collections let programs scale from one record to many records.